

Relazione Basi di dati

**Base di dati scuola di musica
relazione tecnica**

Alberto Bella
Matricola: 1000071543

25 settembre 2026

Indice

1	Introduzione	3
2	Glossario e schemi ER	7
3	Specifiche e vincoli	11
4	Dizionari e tavole	13
5	Valutazione costi e ridondanza	23
6	Traduzioni schemi ER	25
7	Progettazione fisica e indici	29
8	Implementazione fisica delle operazioni	32

Capitolo 1

Introduzione

Introduzione al progetto e analisi delle specifiche Il progetto si prefigge l'obiettivo di costruire una base di dati per una scuola di musica gestendo molteplici scenari:

- Gestione dei corsi individuali e collettivi
- Gestione degli allievi
 - Strumento suonato
 - Livello di preparazione
 - Docente assegnato
 - Orario della lezione
- Tracciamento delle presenze degli allievi
- Esami di livello e saggi

Il database si dovrà inoltre occupare della gestione lato docente con:

- Specializzazioni docenti
- Disponibilità
- Compensi

Capitolo 1 Introduzione

Infine la parte commerciale e amministrativa con:

- Noleggio strumenti musicali
- Vendita spartiti ed accessori
- Aule con strumentazione
- Concerti degli allievi
- Workshop con artisti esterni

Studenti e certificazioni Ogni studente può avere assegnato uno o più strumenti al momento dell'iscrizione nella scuola; di conseguenza, l'assegnazione a un docente per quello stesso strumento in base alla compatibilità degli orari.

Ogni lezione rappresenta uno slot di disponibilità per il docente, il tentativo di inserimento di una lezione all'interno di un slot già precedentemente occupato non è una operazione permessa all'interno del database.

Tutti gli studenti sono classificati in base al loro livello calcolato sulla base di esami conseguiti

In base a ciò sarà possibile determinare quali esami possono effettuare gli studenti (e.g uno studente con livello 2 non può effettuare un esame di livello 4 senza prima aver ottenuto la certificazione per il livello 3)

Tracciamento pagamenti La regolarità amministrativa viene tracciata attraverso il controllo dei regolari pagamenti delle lezioni attraverso una VIEW per visualizzare i pagamenti in ritardo

Aule con strumentazione Le aule destinate alle lezioni presentano dotazioni strumentali eterogenee. In fase di prenotazione da parte di un docente, il sistema dovrà gestire due aspetti fondamentali:

- **Compatibilità strumentazione**

- È necessario che l'aula scelta possieda l'equipaggiamento necessario per lo svolgimento della lezione

- **Esclusività della prenotazione**

- È necessario un vincolo di integrità per impedire sovrapposizioni durante la prenotazione, evitando che la medesima aula venga prenotata contemporaneamente da più docenti.

Noleggio strumenti Il noleggio strumenti richiede l'introduzione di rigorosi vincoli temporali e di disponibilità al fine di prevenire l'incongruenza dei dati quali:

- **Coerenza temporale** Un vincolo di dominio (implementabile attraverso la clausola CHECK) per far sì che la data di fine noleggio non possa essere antecedente a quella di inizio
- **Verifica della disponibilità** Un vincolo che impedisce il noleggio di uno strumento (identificato attraverso una chiave univoca) attualmente già prenotato (attraverso l'opportuno uso di TRIGGER al momento della prenotazione)

Vendita spartiti e accessori Per quanto concerne il modello relativo alla vendita di articoli e accessori (corde, plettri, cavi, etc.) i vincoli si basano sull'integrità della gestione del magazzino e della correttezza delle transazioni, in particolare:

- **Validità dei domini** al momento dell'acquisto di un oggetto è necessario verificare che l'oggetto acquistato sia in una quantità > 0 e l'importo della transazione un numero positivo

Capitolo 1 Introduzione

- **Verifica delle giacenze** Le giacenze in magazzino devono essere gestite attraverso due vincoli fondamentali:
 - Al momento della vendita la quantità dell'oggetto acquistato deve essere pari o superiore a quella in giacenza all'interno del magazzino
 - Una volta evaso l'ordine è necessario un **TRIGGER** che si occupi di decrementare gli oggetti in giacenza in base alla quantità ordinata

Capitolo 2

Glossario e schemi ER

Tabella 2.1: Glossario

Termine	Descrizione	Sinonimo	Legame
Docente	Professionista incaricato di svolgere le lezioni	Insegnante	Aula, Lezione
Aula	Luogo dove si svolgono le lezioni	Classe	Aula, Lezione
Strumento	Attrezzatura musicale che può essere fissa o mobile e noleggiata	-	Noleggio, Lezione
Studente	Persona che segue le lezioni del docente	Allievo	Noleggio, Lezione, Aula, Studente

Tabella 2.1: Glossario

Termine	Descrizione	Sinonimo	Legame
Lezione	Ora di insegnamento dello strumento	Insegnamento	Aula, Lezione, Presenze, Pagamento
Esame	Esame di livello svolto dallo studente	Prova	Studente
Noleggio	Contratto di locazione di uno strumento a tempo	Affitto	Studente, Magazzino
Prodotto	Oggetto da acquistare	Oggetto, Accessorio	Magazzino
Concerto	Attività musicale e dimostrativa	Esibizione, Spettacolo	Studente, Docente
Workshop	Lezione frontale di gruppo con docenti esterni	Laboratorio, Seminario	Aula, Lezione

Capitolo 2 Glossario e schemi ER

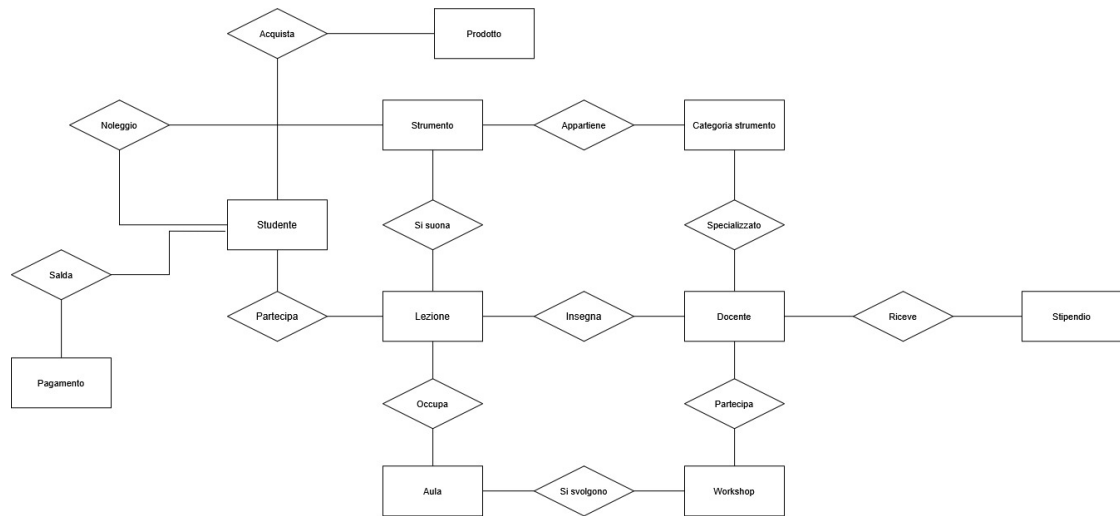


Figura 2.1: Schema Entità-Relazione scheletro

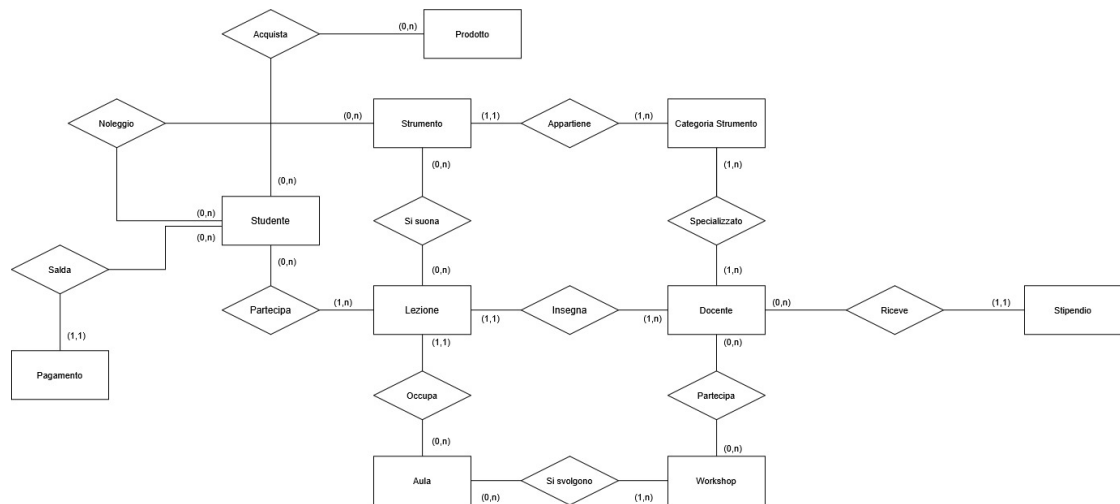


Figura 2.2: Schema Entità-Relazione intermedio

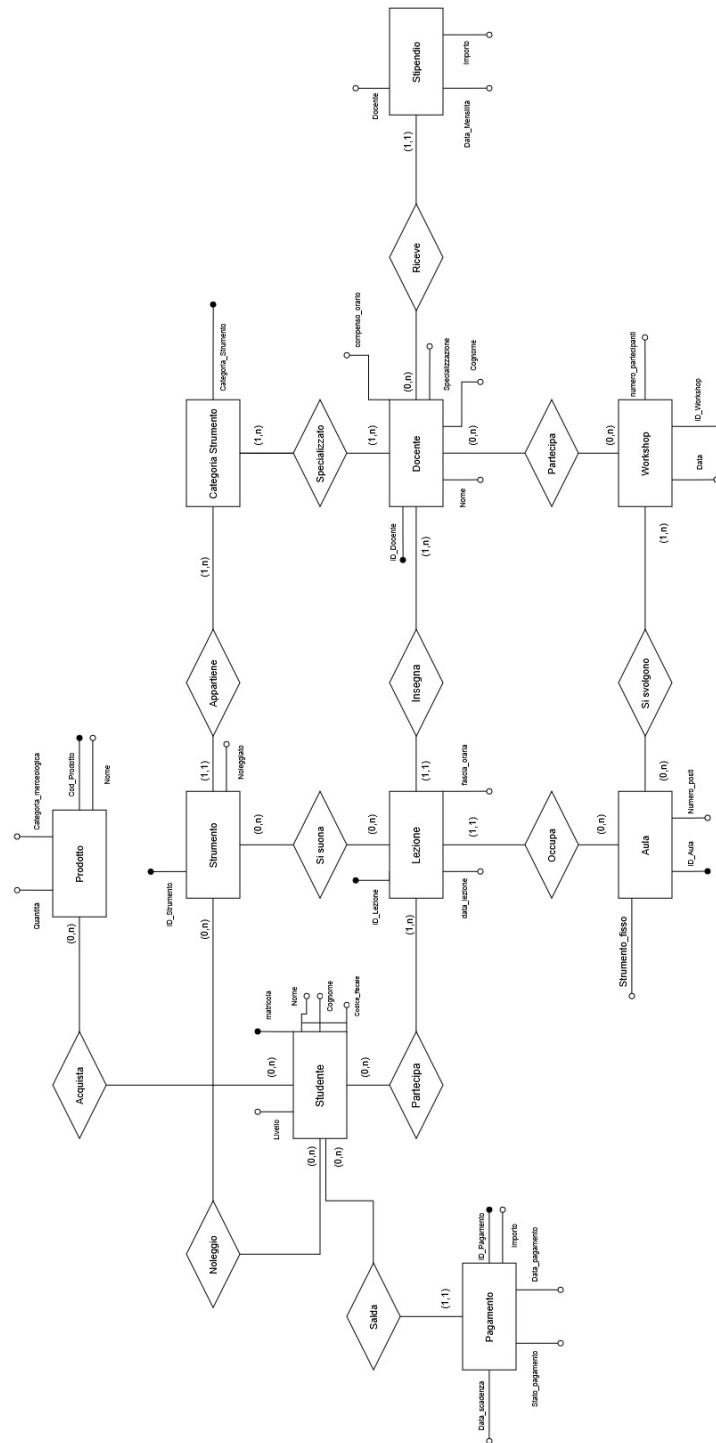


Figura 2.3: Schema Entità-Relazione completo

Capitolo 3

Specifiche e vincoli

Specifiche sulle Operazioni Di seguito un elenco di possibili operazioni che possono essere effettuate all'interno del database:

1. Registrazione nuovo Allievo nella scuola
2. Calcolo stipendi mensili docenti
3. Registrazione di una nuova lezione
4. Acquisto di accessori
5. Aggiornamento prodotti in magazzino
6. Registrazione di nuovi workshop
7. Prenotazione aule
8. Noleggio di strumenti forniti dalla scuola
9. Controllo pagamenti delle lezioni
10. Verifica di disponibilità di tutte le aule in una determinata fascia oraria

Vincoli non esprimibili Di seguito alcuni vincoli non esprimibili nello schema ER:

- Non è possibile registrare lo stesso studente due volte nella scuola
- Non è possibile acquistare un prodotto in magazzino se la quantità selezionata è inferiore a quella in giacenza
- Non è possibile prenotare un'aula se è già occupata per quella fascia oraria
- Non è possibile prenotare un'aula da parte di un docente che usufruisce di uno strumento fisso se l'aula non ne è dotata
- Non è possibile noleggiare uno strumento già noleggiato

Capitolo 4

Dizionari e tavole

Tabella 4.1: Dizionario delle Entità

Entità	Descrizione	Attributi	Identificatore
Pagamento	Pagamenti delle lezioni	ID_pagamento, Data_Pagamento, Importo	ID_Pagamento
Studente	Allievo della scuola di musica	Nome, Cognome, Codice_fiscale, Livello	Codice_Fiscale
Prodotto	Prodotto presente in magazzino	Quantità, Categoria_Merceologica, Cod_Prodotto, Nome	Cod_Prodotto
Strumento	Strumento musicale personale o della scuola	ID_Strumento, Noleggiato	ID_Strumento

Capitolo 4 Dizionari e tavole

Categoria_ Strumento	Categoria dello strumento (e.g a fiato, a corde etc.)	Categoria_ Strumento	Categoria_ Strumento
Docente	Insegnante che svolge lezioni di musica	ID_Docente, Nome, Cognome, Specializzazio- ne	ID_Docente
Workshop	Lezione o lezioni di gruppo con ospiti	Data, ID_Workshop	ID_Workshop
Aula	Luogo dove si svolgono le lezioni	Strumento_fisso, ID_Aula, Numero_Posti	ID_Aula
Lezione	Attività didattica con Docente	Data, ID_Lezione	ID_Lezione
Stipendio	Importo mensile dei docenti	Importo, Data_Mensilità, Docente, Cod_Pagamento	Cod_Pagamento

Tabella 4.2: Dizionario delle Relazioni

Relazione	Partecipanti	Descrizione
Salda	Studente, Pagamento	Lo studente salda il pagamento
Noleggio	Studente, Strumento	Associa gli strumenti noleggiati agli studenti
Acquista	Studente, Prodotto	Lo studente acquista un prodotto dal magazzino
Appartiene	Strumento, Categoria Strumento	Associa ad ogni strumento la sua categoria
Si suona	Strumento, Lezione	Uno strumento si suona a lezione
Occupu	Lezione, Aula	Associa ogni aula a una lezione
Si svolgono	Aula, Workshop	I workshop si svolgono in un'aula
Partecipa	Workshop, docente	Un docente partecipa al workshop
Partecipa	Studente, Lezione	Uno studente partecipa alla lezione
Insegna	Docente, Lezione	Un Docente svolge una lezione

Tabella 4.3: Tavola delle frequenze

Operazione	Descrizione	Frequenza
OP1	Registrazione allievo	2 volte/mese
OP2	Calcolo stipendi	15 volte/mese
OP3	Registrazione nuova lezione	100 volte/mese
OP4	Acquisto di accessori	50 volte/mese
OP5	Aggiornamento prodotti magazzino	70 volte/mese
OP6	Registrazione di nuovi workshop	10 volte/mese
OP7	Prenotazione aule	100 volte/mese
OP8	Noleggio strumenti	10 volte/mese
OP9	Controllo pagamenti lezioni	300 volte/mese
OP10	verifica disponibilità aula	400 volte/mese

Tabella 4.4: Tavola dei volumi, stima dopo un anno

Concetto	Tipo	Volume
Pagamento	E	2000
Studente	E	200
Prodotto	E	20
Strumento	E	350
Categoria Strumento	E	5
Docente	E	20
Workshop	E	50
Aula	E	10
Lezione	E	9246
Stipendio	E	220
Salda	R	1460
Noleggio	R	30
Acquista(Studente, Prodotto)	R	300
Si suona	R	9246
Occupa (Lezione, Aula)	R	9246
Partecipa (Docente, Workshop)	R	40

Capitolo 4 Dizionari e tavole

Partecipa (Studente, Lezione)	R	9246
Insegna (Docente, Lezione)	R	9286

Tabella 4.5: Tavola delle frequenze

Operazione	Descrizione	Frequenza
OP1	Registrazione Allievo	5/mese
OP2	Calcolo stipendi	20/mese
OP3	Registrazione nuova lezione	600/mese
OP4	Acquisto di accessori	200/mese
OP5	Aggiornamento prodotti magazzino	100/mese
OP6	Registrazione nuovi workshop	10/mese
OP7	Prenotazione aule	600/mese
OP8	Noleggio strumenti	20/mese

Capitolo 4 Dizionari e tavole

OP9	Controllo pagamenti	1200/mese
OP10	Verifica disponibilità	1500/mese

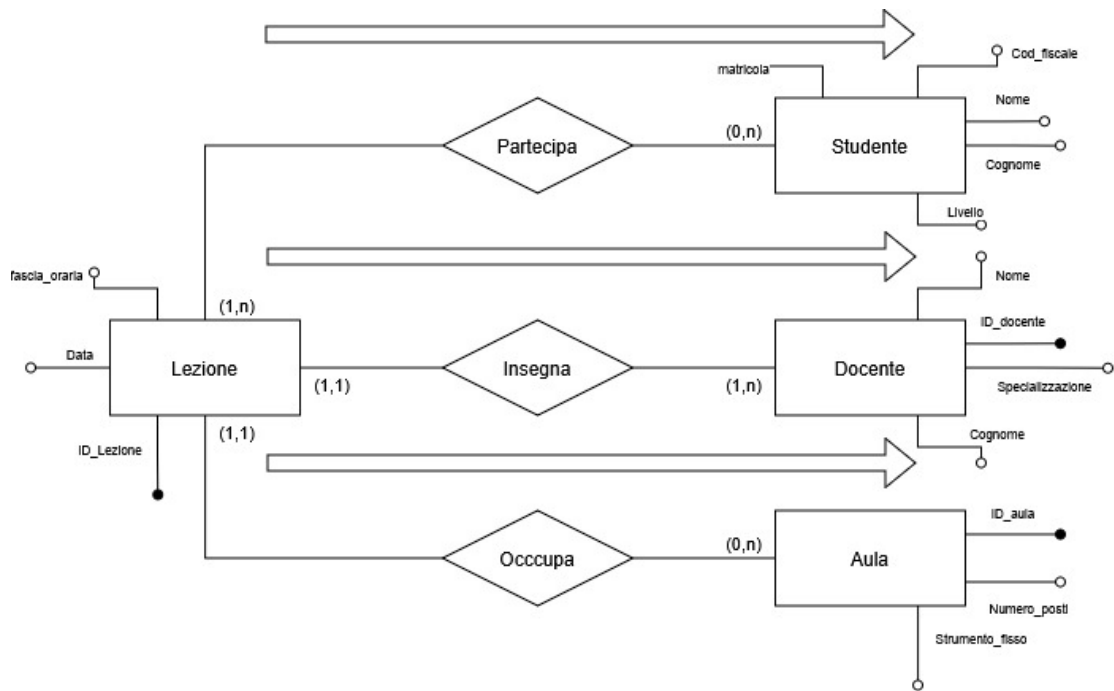


Figura 4.1: Schema operazione OP3

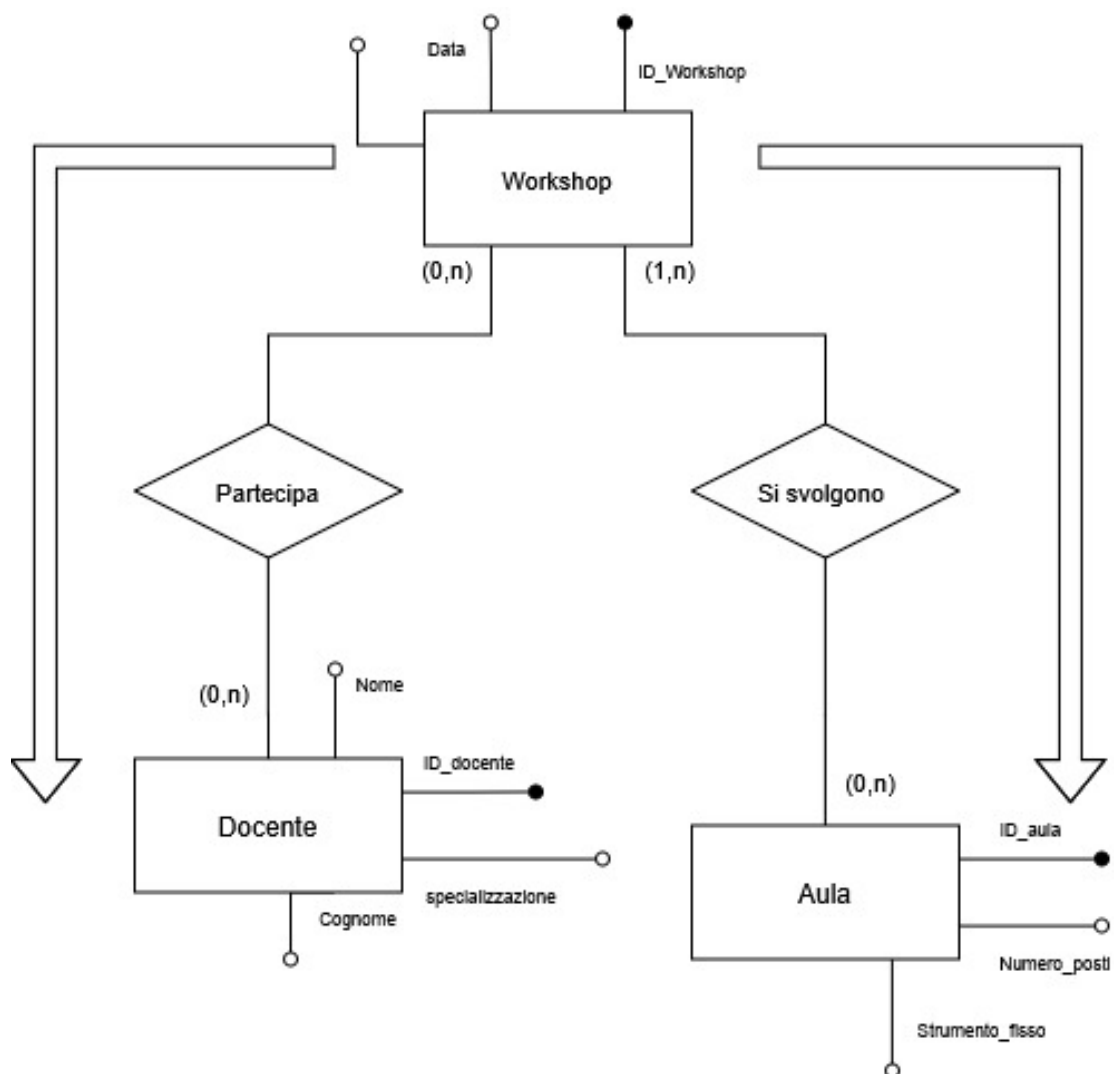


Figura 4.2: Schema operazione OP6

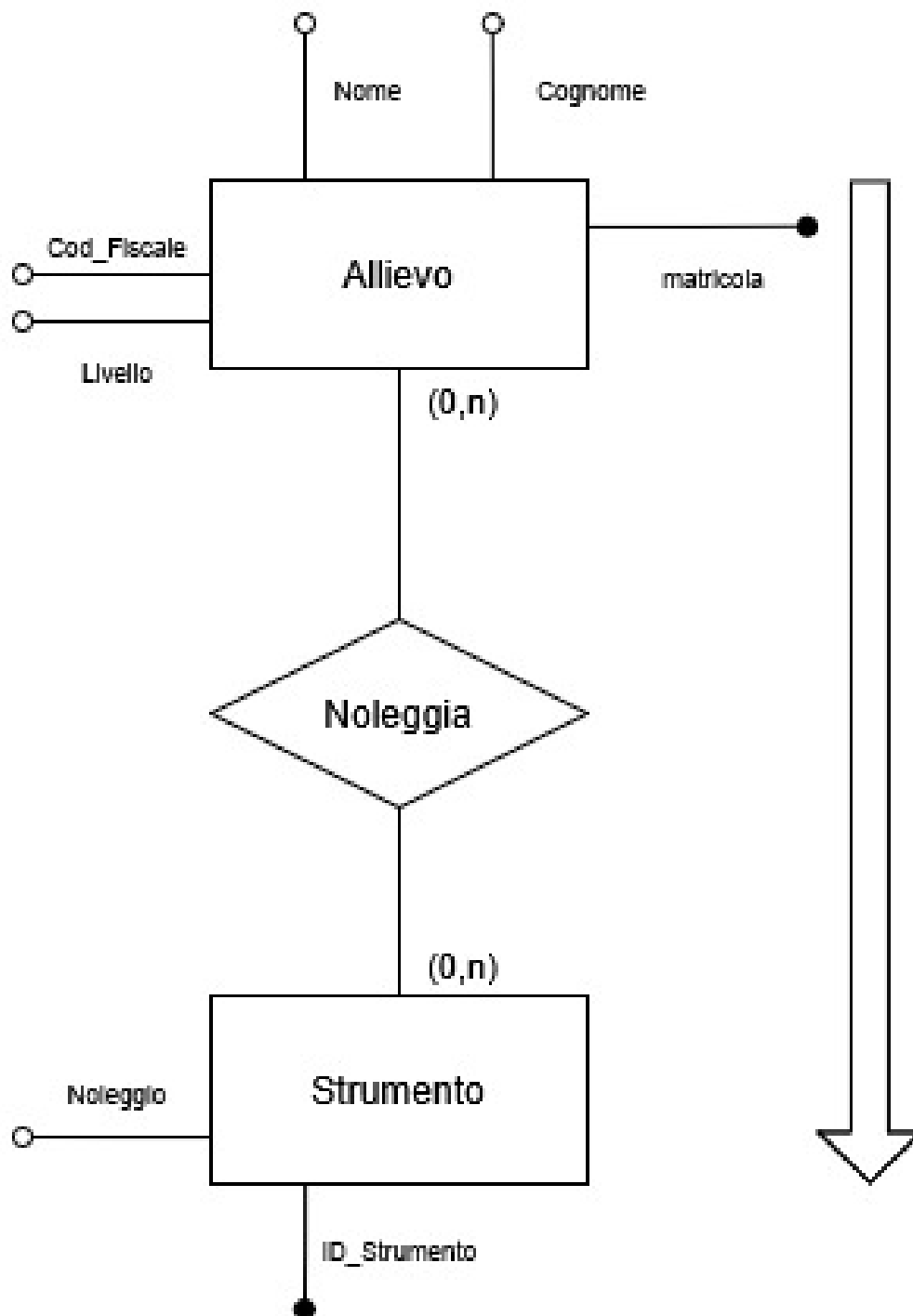


Figura 4.3: Schema operazione OP8

Capitolo 5

Valutazione costi e ridondanza

Attributo ridondante consideriamo due operazioni sull'entità Workshop:

- [OP-A] Verifica posti disponibili (100 volte/mese)
- [OP-B] Iscrizione studenti al workshop (40 volte/mese)

Introduciamo dunque `Numero_partecipanti` al fine di velocizzare le operazioni di lettura all'interno del database, calcoliamo se mantenere l'attributo ridondante sia conveniente o meno:

Con attributo ridondante [OP-A]: 100 letture al mese
[OP-B]: 40 letture al mese + 40 scritture sull'attributo ridondante
Totale: $100 + 40 + (40 \times 2) = 260$ operazioni/mese ¹

Senza attributo ridondante [OP-A] Ipotizzando una media di 8 partecipanti per ogni workshop avremo:

1. 1 lettura su Workshop
2. 1 lettura su `calendario_workshop` (nuova relazione necessaria)
3. 8 conteggi per i partecipanti

¹Considerando ogni scrittura come equivalente a due letture in costo computazionale

Capitolo 5 Valutazione costi e ridondanza

Dunque $10 \times 100 = 1000$ *operazioni/mese*

OP-B 40 scritture in Workshop $\longrightarrow$ 160 *operazioni/mese*

Totale: $1000 + (40 \times 2) = 1160$ operazioni/mese

Conclusione Conviene mantenere l'attributo ridondante considerato il discreto risparmio in termini di costo mensile per le operazioni

Capitolo 6

Traduzioni schemi ER

Traduzioni uno a molti Di seguito consideriamo tutte le relazioni uno a molti del database e le traduciamo in maniera discorsiva:

- La relazione **Occupa** tra le due entità **Aula** e **Lezione** viene inserita nell'entità **Lezione** attraverso la chiave esterna **ID_Aula**
- La relazione **Insegna** tra le due entità **Lezione** e **Docente** viene inserita all'interno dell'entità **Lezione** attraverso la chiave esterna **ID_Docente**
- La relazione **Salda** tra le due entità **Pagamento** e **Studente** viene inserita all'interno dell'entità **Pagamento** attraverso la chiave esterna **Matricola**
- La relazione **Appartiene** tra le due entità **Strumento** e **Categoria Strumento** viene inserita all'interno dell'entità **Strumento** attraverso la chiave esterna **Categoria_Strumento**
- La relazione **Riceve** tra le due entità **Stipendio** e **Docente** viene inserita all'interno dell'entità **Stipendio** attraverso la chiave esterna **ID_Docente**

Traduzioni molti a molti Di seguito le traduzioni delle relazioni molti a molti del database:

- La relazione **Noleggio** tra le due entità **Studente** e **Strumento** viene scorporata in una nuova entità con le chiavi esterne:

1. Matricola
2. ID_Strumento

Inoltre aggiungiamo anche l'attributo **Quantità**

- La relazione **Acquista** tra le due entità **Studente** e **Prodotto** viene scorporata in una nuova entità (Acquisto_Prodotto) con le chiavi esterne:

1. Matricola
2. Cod_Prodotto

Anche qui introduciamo nell'entità l'attributo **Quantità**

- La relazione **Partecipa** tra le due entità **Studente** e **Lezione** viene scorporata in una nuova entità (Partecipazione_Lezioni) con le chiavi esterne:

1. Matricola
2. ID_Lezione

- La relazione **Si svolge** tra le due entità **Aula** e **Workshop** viene scorporata in una nuova entità (Calendario_Workshop) con le chiavi esterne:

1. ID_Aula
2. ID_Workshop

A cui aggiungiamo inoltre l'attributo **data_workshop**

- La relazione **Partecipa** tra le due entità **Workshop** e **Docente** viene scorporata in una nuova entità (*Partecipazione_Docenti_WS*) con le chiavi esterne:
 1. *ID_Docente*
 2. *ID_Workshop*
- La relazione **Specializzato** tra le due entità **Docente** e **Categoria Strumento** viene scorporata in una nuova entità (*Specializzazione_Docenti*) con le chiavi esterne:
 1. *ID_Docente*
 2. *Categoria_Strumento*

Schema logico Ne consegue infine lo schema logico:

- Aula(**ID_Aula**, Strumento_Fisso, Numero_Posti)
- Lezione(**ID_Lezione**, data, *ID_Aula*, *ID_Docente*)
- Docente(**ID_Docente**, nome, cognome, specializzazione, compenso_orario)
- Studente(**matricola**, nome, cognome, codice_fiscale, livello)
- Pagamento(**ID_Pagamento**, data, importo, *matricola*)
- Strumento(**ID_Strumento**, noleggiato, *Categoria_Strumento*)
- Categoria_Strumento(**Categoria_Strumento**)
- Stipendio(**Cod_Pagamento**, data_mensilità, importo, *ID_Docente*)
- Noleggio(**FK_Matricola**, **FK_ID_Strumento**)
- Prodotto(**cod_prodotto**, nome, categoria_merceologica, quantità)
- Acquisto_Prodotto(**FK_Matricola**, **FK_Cod_Prodotto**)
- Partecipazione_Lezioni(**FK_Matricola**, **FK_ID_Lezione**)

- Workshop(**ID_workshop**, data_workshop, numero_partecipanti)
- Calendario_Workshop(**FK_ID_Aula**, **FK_ID_Workshop**, data)
- Partecipazione_Docenti_WS(**FK_ID_Docente**, **FK_ID_Workshop**)
- Specializzazione_Docenti(**FK_ID_Docente**, **Fk_ID_Workshop**)
- Workshop(**ID_Workshop**, data, numero_partecipanti)

Verifica BCNF Il seguente schema logico rispetta per tutte le dipendenze funzionali la forma normale di Boyce-Codd, ad eccezione della entità **Workshop** la quale contiene l'attributo ridondante **Numero_partecipanti** per velocizzare le operazioni nel database

Capitolo 7

Progettazione fisica e indici

Scelta degli indici Oltre all'implementazione di attributi ridondanti per la velocizzazione delle operazioni, si possono implementare anche indici secondari per alcune operazioni molto frequenti, vediamo dunque quali operazioni ho scelto per l'implementazione di un indice secondario:

- **OP10** Verifica disponibilità aula in una determinata fascia oraria, coinvolge in `Lezione` e `Calendario_Workshop`:
 1. `Data`
 2. `Fascia_oraria`
- **OP9** Controllo pagamenti delle lezioni, coinvolge in `Pagamento`:
 1. `Data_scadenza`
 2. `Stato_Pagamento`
- **OP2** Calcolo stipendi mensili docenti, coinvolge in `Lezione`:
 1. `Data`
 2. `ID_Docente`

Di seguito i calcoli per gli indici:

Capitolo 7 Progettazione fisica e indici

OP10 Partendo dai dati:

$$N = 9246$$

$$B = 4096 \text{ byte}$$

$$R = 50 \text{ byte}$$

$$P = 4 \text{ byte}$$

$$V = 8 \text{ byte}$$

$$bfr = \lfloor \frac{B}{R} \rfloor = \lfloor \frac{4096}{50} \rfloor = 81 \text{ blocchi}$$

$$N_b = \lceil \frac{N}{bfr} \rceil = \lceil \frac{9246}{81} \rceil = 115 \text{ blocchi}$$

$$E_i = V + P = 8 + 4 = 12 \text{ byte}$$

$$bfr_i = \lfloor \frac{B}{E_i} \rfloor = \lfloor \frac{4096}{12} \rfloor = 341 \text{ blocchi}$$

$$\text{Calcolo indice denso: } N_b = N = 9246 \quad b_{denso} = \lceil \frac{N}{bfr_i} \rceil = \lceil \frac{9246}{341} \rceil = 28$$

$$\text{Costo di ricerca: } \lceil \log_2 b_{denso} \rceil + 1 = \lceil \log_2 28 \rceil + 1 = 5 + 1 = 6 \text{ accessi}$$

OP9 Partendo dai dati

$$N = 2000$$

$$B = 4096 \text{ byte}$$

$$R = 50 \text{ byte}$$

$$P = 4 \text{ byte}$$

$$V = 26 \text{ byte}$$

$$bfr = \lfloor \frac{B}{R} \rfloor = \lfloor \frac{4096}{50} \rfloor = 81 \text{ blocchi}$$

$$N_b = \lceil \frac{N}{bfr} \rceil = \lceil \frac{2000}{81} \rceil = 25 \text{ blocchi}$$

$$E_i = V + P = 26 + 4 = 30 \text{ byte}$$

$$bfr_i = \lfloor \frac{B}{E_i} \rfloor = \lfloor \frac{4096}{30} \rfloor = 136 \text{ blocchi}$$

$$\text{Calcolo indice denso: } N_b = N = 2000 \quad b_{denso} = \lceil \frac{N}{bfr_i} \rceil = \lceil \frac{2000}{136} \rceil = 15$$

$$\text{Costo di ricerca: } \lceil \log_2 b_{denso} \rceil + 1 = \lceil \log_2 15 \rceil + 1 = 4 + 1 = 5 \text{ accessi}$$

OP2 Partendo dai dati:

$$N = 9246$$

$$B = 4096 \text{ byte}$$

$$R = 50 \text{ byte}$$

$$P = 4 \text{ byte}$$

$$V = 8 \text{ byte}$$

$$bfr = \lfloor \frac{B}{R} \rfloor = \lfloor \frac{4096}{50} \rfloor = 81 \text{ blocchi}$$

$$N_b = \lceil \frac{N}{bfr} \rceil = \lceil \frac{9246}{81} \rceil = 115 \text{ blocchi}$$

$$E_i = V + P = 8 + 4 = 12 \text{ byte}$$

$$bfr_i = \lfloor \frac{B}{E_i} \rfloor = \lfloor \frac{4096}{12} \rfloor = 341 \text{ blocchi}$$

$$\text{Calcolo indice denso: } N_b = N = 9246 \quad b_{denso} = \lceil \frac{N}{bfr_i} \rceil = \lceil \frac{9246}{341} \rceil = 28$$

$$\text{Costo di ricerca: } \lceil \log_2 b_{denso} \rceil + 1 = \lceil \log_2 28 \rceil + 1 = 5 + 1 = 6 \text{ accessi}$$

Notiamo come il risultato del terzo indice coincide con il primo

Capitolo 8

Implementazione fisica delle operazioni

Implementazione delle operazioni di seguito le 10 operazioni descritte prima in SQL:

[OP1] registrazione nuovo allievo nella scuola

```
INSERT INTO Studente(matricola, nome, cognome, codice_fiscale,  
    livello)  
VALUES (893092, 'Giovanni', 'Crescenzi', 'gvncrsn04p18c351k', 2);
```

[OP2] calcolo stipendi mensili docenti

```
SELECT
    L.ID_Docente,
    D.nome,
    D.cognome,
    MONTH(L.data_lezione) AS mese,
    YEAR(L.data_lezione) AS Anno,
    COUNT(L.ID_Lezione) AS numero_lezioni,
    (COUNT(L.ID_Lezione) * D.compenso_orario) AS
        stipendio_mensile
FROM Lezione L
JOIN Docente D ON L.ID_Docente = D.ID_Docente
GROUP BY
    L.ID_Docente,
    D.nome,
    D.cognome,
    YEAR(L.data_lezione),
    MONTH(L.data_lezione);
```

[OP3] registrazione di una nuova lezione

```
INSERT INTO Lezione(ID_Lezione, data_lezione, ID_Aula, ID_Docente,
    fascia_oraria)
VALUES (28939, '18-12-2025', 18, 4898, '18:00:00');
```

[OP4] acquisto di accessori

```
INSERT INTO Acquisto_prodotto(FK_matricola, FK_cod_prodotto,
    quantit)
VALUES (893092, 181911, 3);
```

[OP5] aggiornamento prodotti in magazzino

```
DELIMITER //
CREATE TRIGGER aggiornamento_quantit_magazzino
AFTER INSERT ON Acquisto_prodotto
FOR EACH ROW
BEGIN
    UPDATE Prodotto
    SET quantit = quantit - NEW.quantit
    WHERE cod_prodotto = NEW.FK_cod_prodotto;
END //
```

[OP6] registrazione di nuovi workshop

```
INSERT INTO Workshop(ID_Workshop, data_workshop,
    numero_partecipanti)
VALUES (1928, '19-10-2026', 12);
```

[OP7] prenotazione aule

```
INSERT INTO Lezione(ID_Lezione, data_lezione, ID_Aula, ID_Docente,
    fascia_oraria)
VALUES (12493, '24-03-2026', 12, 4898, '15:00:00');
-- o in alternativa per il workshop
INSERT INTO Calendario_Workshop(FK_ID_Aula, FK_ID_Workshop,
    data_workshop, fascia_oraria)
VALUES(14, 19109, '07-02-2026', '19:00:00');
```

[OP8] noleggio strumenti forniti dalla scuola

```
INSERT INTO Noleggio(FK_matricola, FK_ID_Strumento, data_inizio,
    data_fine)
VALUES (18930, 48930, '19-02-2026', '20-03-2027');
```

[OP9] controllo pagamenti delle lezioni

```
CREATE VIEW lista_pagamenti_in_ritardo
SELECT S.nome, S.cognome, S.matricola, P.ID_pagamento,
       P.data_scadenza, P.stato_pagamento, P.importo
FROM Studente S Join Pagamento P on S.matricola = P.matricola
WHERE P.stato = 'non pagato' and P.data_scadenza < CURRENT_DATE();
```

[OP10] verifica disponibilità di tutte le aule in una fascia oraria

```
DELIMITER //
CREATE PROCEDURE verifica_disponibilita_aula (IN param_data, IN
       param_fascia_oraria)
BEGIN
    SELECT A.ID_Aula
    FROM Aula A
    WHERE A.ID_Aula NOT IN (
        SELECT ID_Aula
        FROM Lezione
        WHERE data_lezione = param_data AND fascia_oraria =
            param_fascia_oraria;
    )
    AND NOT IN (
        SELECT ID_Aula
        FROM Calendario_Workshop
        WHERE data_workshop = param_data AND fascia_oraria =
            param_fascia_oraria;
    )
END //

CALL verifica_disponibilita_aula('28-11-2026', '15:00:00');
```

Implementazione fisica dei trigger di seguito l'implementazione fisica dei trigger in SQL:

[T1] Non è possibile acquistare un prodotto in quantità superiore alla giacenza, con importo < 0 oppure quantità < 0

```
DELIMITER //
```

```
CREATE TRIGGER acquisto_fuori_giacenza  
BEFORE INSERT ON Acquisto_Prodotto  
FOR EACH ROW  
BEGIN  
    DECLARE giacenza_prodotto NUMBER;  
  
    SELECT quantit INTO giacenza_prodotto  
    FROM Prodotto P  
    WHERE P.cod_prodotto = NEW.FK_cod_prodotto;  
  
    IF N.importo < 0 OR N.quantit < 0 OR N.quantit >  
        giacenza_prodotto THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'impossibile concludere  
            l''acquisto, il prodotto supera la giacenza in  
            magazzino, l''importo <0 oppure la quantita <0';  
    END IF;  
END //
```

[T2] Non è possibile prenotare l'aula se è già occupata per quell'orario

```
CREATE TRIGGER prenotazione_aula_lezione
BEFORE INSERT ON Lezione
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Lezione L
        WHERE L.ID_Aula = NEW.ID_Aula AND L.data_lezione =
            NEW.data_lezione AND L.fascia_oraria =
            NEW.fascia_oraria;
    )
    OR EXISTS (
        SELECT 1
        FROM Calendario_Workshop W
        WHERE W.FK_ID_aula = NEW.ID_Aula AND W.data_workshop
            = NEW.data_lezione AND W.fascia_oraria =
            NEW.fascia_oraria;
    )
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'fascia oraria non disponibile
            per la prenotazione';
    END IF;
END //
```

[T2.1] Idem al precedente per i workshop

```
CREATE TRIGGER prenotazione_aula_workshop
BEFORE INSERT ON Calendario_Workshop
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Lezione L
        WHERE L.ID_Aula = NEW.ID_Aula AND L.data_lezione =
            NEW.data_lezione AND L.fascia_oraria =
            NEW.fascia_oraria;
    )
    OR EXISTS (
        SELECT 1
        FROM Calendario_Workshop W
        WHERE W.FK_ID_aula = NEW.ID_Aula AND W.data_workshop
            = NEW.data_lezione AND W.fascia_oraria =
            NEW.fascia_oraria;
    )
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'fascia oraria non disponibile
            per la prenotazione';
    END IF;
END //
```

[T3] Non è possibile prenotare una lezione che richiede lo strumento fisso se l'aula ne è sprovvista

```
CREATE TRIGGER prenotazione_strumento_fisso
BEFORE INSERT ON Lezione
FOR EACH ROW
BEGIN
    DECLARE dotazione_strumento_fisso BOOLEAN;
    DECLARE specializzazione_docente VARCHAR(60);

    SELECT Strumento_fisso INTO dotazione_strumento_fisso
    FROM Aula A
    WHERE N.ID_Aula = A.ID_Aula;

    SELECT specializzazione INTO specializzazione_docente
    FROM Docente D
    WHERE NEW.ID_Docente = D.ID_Docente;

    IF specializzazione_docente = 'batteria' OR
       specializzazione_docente = 'pianoforte' AND
       dotazione_strumento_fisso = FALSE THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Aula sprovvista di strumento
                           fisso, impossibile prenotare!'
    END IF;
END //
```

[T4] Non è possibile noleggiare uno strumento già noleggiato

```
CREATE TRIGGER noleggio_strumento
BEFORE INSERT ON Noleggio
FOR EACH ROW
BEGIN
    DECLARE strumento_noleggiato BOOLEAN;

    SELECT noleggiato INTO strumento_noleggiato
    FROM Strumento S
    WHERE S.ID_Strumento = NEW.FK_ID_strumento;

    IF strumento_noleggiato THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Lo strumento scelto gi stato
                             noleggiato!'
    END IF;
END //
```

[T5] non è possibile saltare un livello per lo studente

```
CREATE TRIGGER avanzamentolivello_studente
BEFORE UPDATE ON INSERT
FOR EACH ROW
BEGIN
    IF NEW.livello > OLD.livello + 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Non possibile saltare un
                             livello per lo Studente'
    END IF;
END //
```

Conclusioni e uso strumenti Il seguente progetto è stato creato attraverso gli strumenti:

- Overleaf per la scrittura della relazione in pdf
- Notepad++ per la scrittura del codice SQL
- Gemini 3.1 Pro per alcune specifici prompt legati alle seguenti attività:
 1. Aiuto con la formattazione del codice $\text{\LaTeX}$ per le tabelle del glossario e dizionario dati
 2. Creazione del dump del database

Il codice SQL (ad eccezione del dump) **NON** è stato generato dall'intelligenza artificiale